

Cybersecurity Internship – Week 1 Report

Security Assessment of Web Application(JUICE SHOP)

Student Information

- **Name:** Syed Muhammad Talal
- **Intern ID:** DHC-476
- **Organization:** Developer Hub Corporation

Objective

The objective of this task was to perform a basic security assessment of a web application by identifying common vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection using manual tools

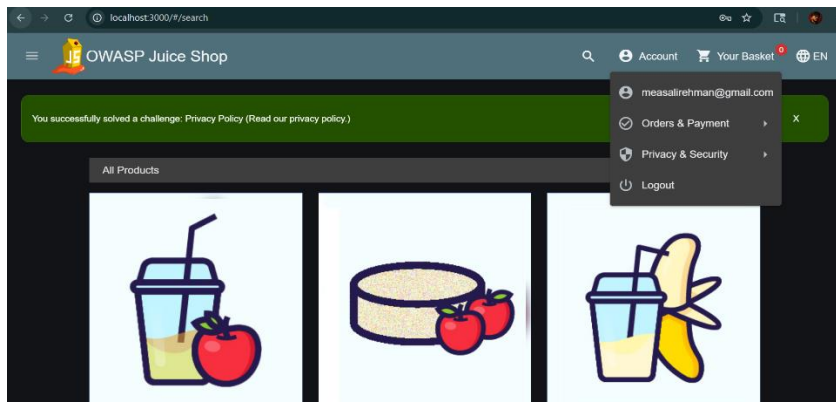
1. Application Setup & Exploration:

A mock vulnerable web application was set up locally using Node.js.

Steps Performed:

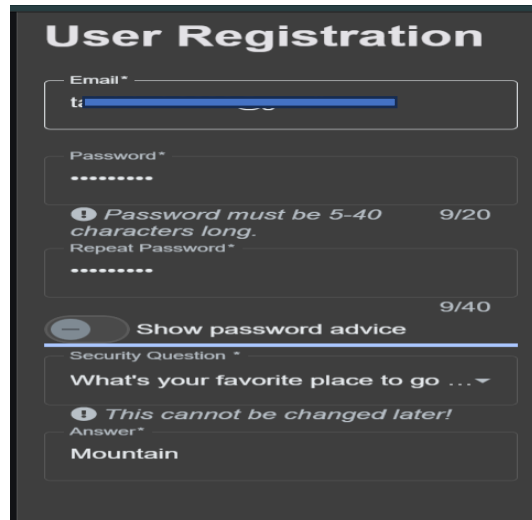
- Installed dependencies using npm install
- Started application using npm start
- Accessed application at: <http://localhost:3000>
- Explored key functionalities:
 - User Signup
 - User Login
 - Product and profile-related features

SCREEN OUTPUTS:



Home page

- **Registration/Sing-up page**

A screenshot of a 'User Registration' form. The form has a dark background with white text. It includes fields for 'Email *' (containing 't'), 'Password *' (masked with dots), and 'Repeat Password *' (also masked). A password strength indicator shows 'Password must be 5-40 characters long.' and '9/20'. Below these is a 'Show password advice' toggle switch. The 'Security Question *' is 'What's your favorite place to go ...' with a dropdown arrow. A warning message states 'This cannot be changed later!'. The 'Answer *' field contains the word 'Mountain'.

- **User Login**

Observation:

The application was successfully deployed and functional in a local environment.

2. Cross-Site Scripting (XSS) Testing

Scripting used :

```
<script>alert('XSS')</script>
```

Testing Method:

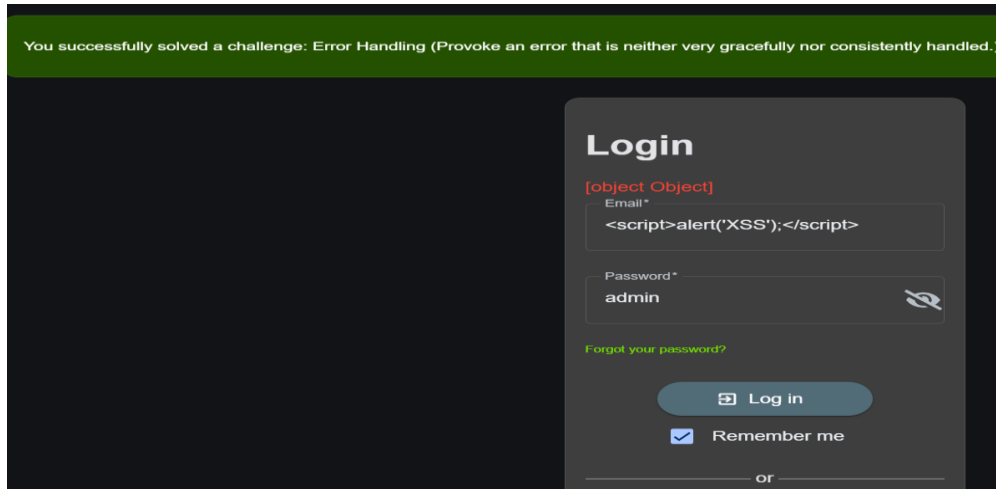
The script was entered into input fields such as username and signup forms.

Result:

- The application did not execute the script
- No pop-up alert happen
- Instead, a system-generated challenge message appeared

Conclusion:

The application appears to have input sanitization or filtering mechanisms in place, preventing basic XSS execution.



3. SQL Injection Testing

Payload Used:

admin' OR '1'='1

Testing Method:

The payload was entered into both username and password fields on the login page.

Result:

- Login attempt failed
- Server returned **401 Unauthorized**

Conclusion:

The application is protected against basic SQL injection attacks, likely through parameterized queries or ORM-based validation.

4. Browser Developer Tools Analysis

Tool Used:

Browser Developer Tools (Network Tab)

Observations:

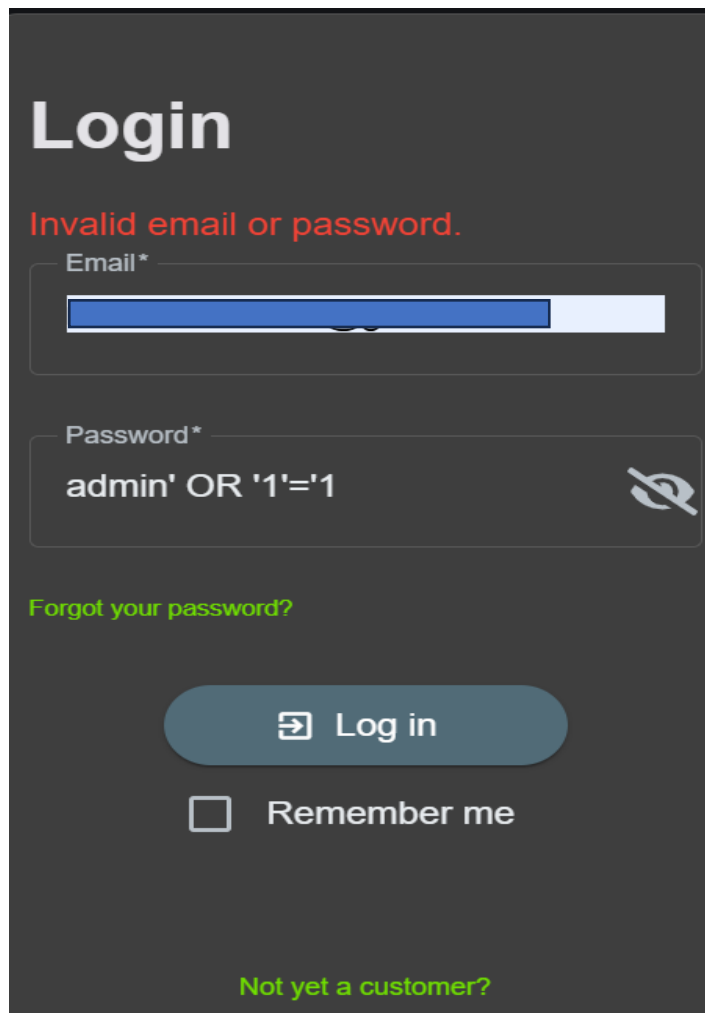
- Login request endpoint: /rest/user/login
- Response code: **401 Unauthorized**

- Additional endpoint observed: /rest/user/whoami
- Response code: **304 Not Modified**
- No authentication tokens or sensitive data exposed

Conclusion:

The application correctly handles authentication and does not expose sensitive information through API responses.

Authenticity check



The screenshot shows a dark-themed login interface. At the top, the word "Login" is displayed in large white text. Below it, a red error message reads "Invalid email or password.". There are two input fields: "Email*" and "Password*". The "Email*" field is empty. The "Password*" field contains the text "admin' OR '1'='1" and has a toggle icon (an eye with a slash) to its right. Below the password field, there is a green link that says "Forgot your password?". A blue "Log in" button with a white icon is positioned below the "Forgot your password?" link. Underneath the button is a checkbox labeled "Remember me". At the bottom of the form, there is a green link that says "Not yet a customer?".

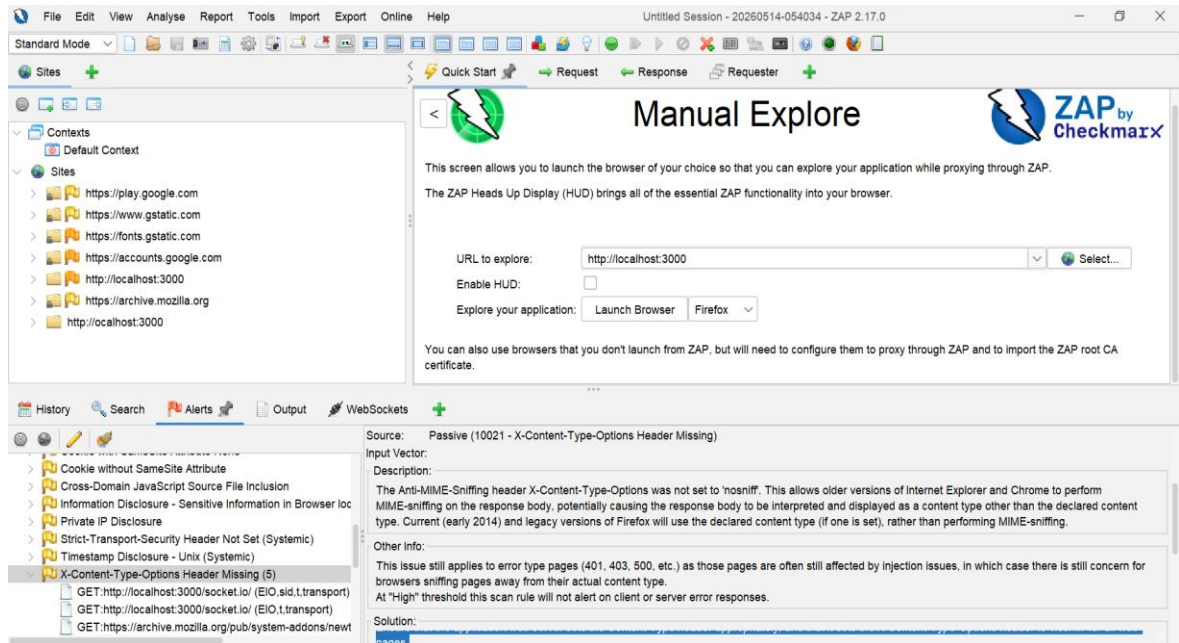
5. OWASP ZAP Passive Scan

Tool Used:

OWASP ZAP (Standard Mode – Passive Scanning)

Method:

- Application traffic captured through proxy
- Manual browsing performed
- Passive scan executed on captured requests



Key Findings:

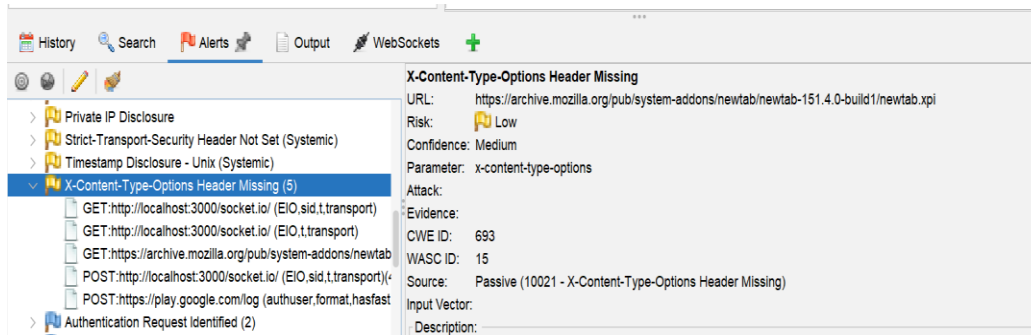
- Missing Security Headers:
 - Content Security Policy (CSP)
 - X-Content-Type-Options
 - Strict-Transport-Security
- Information Disclosure:
 - Server version information exposed
 - Private IP disclosure
- Session Management Issue:
 - Session ID found in URL
- Additional Issues:
 - Cache-related warnings

- Timestamp disclosure

Risk Level:

- No High-risk vulnerabilities identified
- Mostly Low to Medium risk issues detected

Alerts:



Overall Conclusion:

The application demonstrates a reasonable level of security against basic attack techniques such as XSS and SQL injection. However, several security misconfigurations and information disclosure issues were identified during passive scanning. These findings highlight areas where security can be improved, particularly in implementing proper HTTP security headers and securing session management.

Recommendations

- Implement strong Content Security Policy (CSP)
- Enable security headers such as:
 - X-Frame-Options
 - X-Content-Type-Options
 - Strict-Transport-Security
- Avoid exposing server and internal system information
- Improve session handling mechanisms (avoid session IDs in URLs)